Can you help me design an app (it will use R Shiny, but that probably doesnt matter for now). The app should show a tree and allow the user to select different parts, showing different nooks and crevices. These all provide habitats for different animals. I am working with a scientist who has a list of species for these types of habitats within a tree. They would like it to be possible to select any mumber of sections of a tree (some go down multiple sub categorrie, and it should be possible to select top level categories or lower sub categories), and see which species could live there. They also like it to work the other way round, so users can specify species and see what habitats within a tree that they prefer.

For now I cant fully visualise the based way to set up the UI and to provide a good UX. Can you help?

You said

Does using bslib mean it wont be an rshiny app? I want this to be easy to publish online.

You said

I already have the table of data. Each row is a species, and each column is a set of habitat descriptions and preferences for a range of tree species. Can you show me some visuals of how you rhink this 2 columns ui could look and work?

Gemini said

Since your data is structured with species as rows and habitat preferences as columns, the UI needs to act like a dynamic "matcher."

Using `bslib`, we can create a sleek, two-column layout that feels like a professional research tool. The goal is to avoid a "spreadsheet look" and instead create a **tactile experience.**

The "Symmetric" Layout

I recommend a **split-pane approach**. One side represents the **Physical Tree** (Habitats), and the other represents the **Biological Community** (Species).

Column 1: The Habitat "Deep Dive"

Instead of simple checkboxes, use a **Nested Accordion** or a **Tree View**.

- **Top Level:** Root, Trunk, Branch, Canopy.
- **Mid Level:** Bark, Hollows, Leaves.
- **Sub Level:** Deep Fissures, Rotting Heartwood, Lichen Cover.
- **Interaction:** When a user selects "Deep Fissures," the Species column instantly shrinks to show only the creatures the scientist has tagged for that specific column.

Column 2: The Species "Gallery"

Instead of a boring table, use **Species Cards**.

- **The Card:** Each card shows the species name, a small thumbnail (if available), and "tags" that correspond to its habitat preferences.
- **Interaction:** If a user clicks a card (e.g., *Stag Beetle*), the Habitat column on the left should **auto-highlight** or "glow" to show exactly which parts of the tree it occupies.

Visualizing the Bidirectional Flow

To make this work in Shiny, you essentially create two "filters" that talk to each other.

Feature	Habitat → Species	Species → Habitat
User Action	Clicks "Hollows" on a tree diagram.	Searches "Owl" in a search bar.
Visual Result	Species list filters to Woodpeckers, Owls, Bats.	The "Trunk" and "Large Branch" sections of the tree highlight.
R Logic	`filter(data, habitat_col == "Hollow")`	`data[data$Species == "Owl", habitat_cols]`

Pro-Level UX Ideas

1. **The "Ghosting" Effect:** When a species is selected, don't just show its primary habitat. Use different shades to show "Preferred" vs. "Occasional" habitats based on the scientist's data.
2. **Mobile Responsiveness:** Because you're using `bslib`, if someone opens this on a phone in the forest, the two columns will stack vertically—Habitat on top, Species on bottom—making it a perfect field guide.
3. **Breadcrumbs:** As they go deeper into the "nooks and crevices," show a trail like:
   Trunk > **Bark** > **Deep Fissure**

Next Step

To build this, we need to map your column names to the UI. **Would you like to share a few of your column names and a couple of example species?** I can then write the specific R code to turn those columns into the nested navigation menu.

You said

I want to stick to UI design ideas for now, not code. Can you show my an image of how this ui could work?

Gemini said



You said

This is a great start. Would we be able to change our select multiple tree species? And would it be possible for the ui to zoom in on the relevant part of the tree when the user clicks?

Gemini said



You said

Please can you write out plan for this app? Broad overview of R-shiny code requirements, and the design of the UI?

Gemini said

This plan outlines a high-level architecture for your R Shiny app. By combining `bslib` for the layout and `shinyjs` for interactive effects like zooming, we can create a tool that feels more like a modern website than a standard data dashboard.

---

1. UI Design: The "Split-Logic" Interface

The UI will be built using a `page_sidebar()` layout. This keeps controls accessible while giving the interactive elements center stage.

- **Global Filters (Sidebar):** * **Tree Species Multi-Select:** A `selectizeInput` with `multiple = TRUE`. This allows the scientist to compare habitat data across different tree types (e.g., "Oak" AND "Beech").
    - **Habitat Hierarchy:** A nested `accordion()` from `bslib`. Top-level panels like **Trunk** will open to reveal checkboxes for **Bark**, **Hollows**, and **Sapwood**.
- **The Main Stage (Two Columns):**
    - **Left Column (The Visual Tree):** An interactive image container. When a user clicks a "nook" (like a hollow), the app uses CSS transforms or `renderImage` logic to **zoom in** on that coordinate.
    - **Right Column (The Species Gallery):** A dynamic grid of cards. Selecting a species here will "light up" the corresponding habitat parts on the tree image using an SVG overlay.

---

2. Code Requirements & Structure

To make this work in R Shiny, you will need a few specific "engine parts":

A. The Data Engine

Your "Species x Habitat" table should be loaded into a **Reactive Value**.

- **Logic:** When the Tree Species or Habitat checkboxes change, a `reactive()` function filters the master table.
- **Bidirectional Link:** You will use `observeEvent` to watch the Species selection. If a user selects a species, the code "looks up" its preferred columns and updates the sidebar checkboxes automatically using `updateAccordion()`.

B. The Interactive Zoom

Since you want the UI to "zoom in," you have two main R-friendly paths:

1. **CSS Zoom (The "Sleek" Way):** Use `shinyjs::runjs()` to trigger a CSS `transform: scale(2)` on the image container, centered on the coordinates of the click.
2. **SVG Mapping (The "Scientist" Way):** Use an SVG of a tree where each "nook" is a distinct path. When clicked, Shiny captures the `path_id` and filters the data.

C. Necessary Libraries

- `shiny` **&** `bslib`**:** Core framework and modern UI components.
- `shinyjs`**:** Essential for the "zoom" and "highlight" animations.
- `htmlwidgets`**:** If you decide to use a D3-based sunburst for the hierarchy.
- `reactable` **or** `DT`**:** For an optional "Data View" tab for the scientist.

---

3. High-Level App Plan

Phase	Task	UX Focus
1. Data Setup	Map your "Columns" to a hierarchy (e.g., Column A = Level 1, Column B = Level 2).	Ensure data joins are fast.
2. UI Shell	Build the `bslib` sidebar with the multi-select tree species list.	Make it mobile-responsive.
3. Interactive Tree	Create the clickable tree image with "coordinates" for each habitat.	Zooming should feel smooth.
4. Reactive Logic	Write the "bidirectional" filters (Habitat ↔ Species).	Avoid infinite loops in reactivity.
5. Species Gallery	Design the result cards with species images and habitat tags.	Visual feedback on match.

Suggested Navigation Flow

1. **Select Tree(s):** User selects "Oak" and "Pine."
2. **Explore Structure:** User clicks the "Trunk" on the tree image ⟶ UI zooms into the bark texture.
3. **Refine Search:** User selects "Deep Fissures" in the sidebar.
4. **View Results:** The Species gallery on the right updates to show only insects and fungi found in Oak/Pine bark fissures.

Would you like me to focus on the logic for the "Zoom on Click" feature or the "Bidirectional Filter" logic first?